数学 Math/

> 依赖 Template/ihxnan.hpp 的类型别名（ ll 、 vi 、 lll = __int128 等）。 **冲突提醒：** MillerRabin.hpp 与 PollarRho.hpp 定义了相同符号（全局 Bases 、 mul 、 qmi 、 is_prime ），**不能同时粘贴；** PollarRho 是 MillerRabin 的超集（多出 factorize ），需要分解质因数时只用 PollarRho。 **全局状态：** EulerPrime 声明全局 vector<int> prime 与 vector<bool> is_prime 。

组合数（阶乘 + 逆元，按需扩表） Comb.hpp

- **用法**：`Comb comb(998244353);` 自行实例化（传入模数）；`comb.C(n, m)` 组合数、`comb.P(n, m)` 排列数、`comb.fac(x)` / `comb.ifac(x)` 阶乘与阶乘逆元、`comb.inv(x)` 单点逆元。
- **复杂度**：`init` 摊还 `O(n)`，单次查询 `O(1)`（`inv` 单次 `O(log mod)`）。
- **注意**：`C` / `P` 越界参数（`x < 0`、`y < 0`、`x < y`）返回 `0`；`fac` / `ifac` 首次用到才扩表；模数须为质数（费马小定理求逆元）。

```
struct Comb // 组合数（阶乘 + 逆元，按需扩表）
{
    int n;                      // 当前已预处理到的上界
    ll mod;                     // 模数（须为质数）
    vector<ll> _fac, _inv;      // 阶乘表 _fac[i]=i!，逆元表 _inv[i]=(i!)^{-1}

    Comb(ll mod) : n(0), mod(mod), _fac{1}, _inv{1}
    {
    }

    ll qmi(ll a, ll b) // 快速幂
    {
        ll ans = 1;
        for (; b; b >>= 1, a = a * a % mod)
            if (b & 1)
                ans = ans * a % mod;
        return ans;
    }

    void init(int m) // 扩表到 m（增量计算）
    {
        if (m <= n)
            return;
        _fac.resize(m + 1);
        _inv.resize(m + 1);
        for (int i = n + 1; i <= m; ++i)
            _fac[i] = _fac[i - 1] * i % mod;
        _inv[m] = qmi(_fac[m], mod - 2);              // 阶乘逆元
        for (int i = m; i > n; --i)                   // 倒推各阶乘逆元
            _inv[i - 1] = _inv[i] * i % mod;
        n = m;
    }

    ll fac(int x) // x!（自动扩表）
    {
        if (x > n)
            init(x);
        return _fac[x];
    }

    ll inv(ll x) // 单点逆元（费马小定理）
    {
        return qmi(x, mod - 2);
    }

    ll ifac(int x) // (x!)^{-1}
    {
        if (x > n)
            init(x);
        return _inv[x];
    }

    ll C(int x, int y) // 组合数 C(x, y)，非法输入返回 0
    {
```

```
        if (x < 0 || y < 0 || x < y)
            return 0;
        return fac(x) * ifac(y) % mod * ifac(x - y) % mod;
    }

    ll P(int x, int y) // 排列数 P(x, y)
    {
        if (x < 0 || y < 0 || x < y)
            return 0;
        return fac(x) * ifac(x - y) % mod;
    }
};
```

欧拉线性筛（素数表） EulerPrime.hpp

- **用法**：`euler_prime(N);` 之后用全局 `prime`（升序素数表）与 `is_prime[0..N]`（素性）。
- **复杂度**：`O(n)`。
- **注意**：`prime` / `is_prime` 为**全局变量**，多次调用会追加/扩容而非重置；最小质因子性质：每个合数只被其最小质因子筛掉。

```
vector<int> prime;     // 升序素数表
vector<bool> is_prime; // is_prime[i]: i 是否为素数

void euler_prime(int n)
{
    is_prime.resize(n + 1, true);
    for (int i = 2; i ⩽ n; ++i)
    {
        if (is_prime[i])
            prime.push_back(i);
        for (auto &p : prime)
        {
            if (p * i > n)
                break;
            is_prime[i * p] = false;   // 用最小质因子 p 筛掉 i*p
            if (i % p == 0)            // p 是 i 的最小质因子，停止
                break;
        }
    }
}
```

扩展欧几里得（解不定方程） Exgcd.hpp

- **用法**：**代码片段**，整体粘贴进 solve() 内（clac 是捕获 [&] 的 lambda）。clac(a, b, c) 解 $a \cdot x + b \cdot y = c$。
- **输出**：有**正整数解**时输出 5 行——解的个数、x 最小值、y 最小值、x 最大值、y 最大值；**无正整数解**时输出 2 行——x 最小正整数值、y 最小正整数值；c % gcd != 0 输出 -1。
- **注意**：exgcd 返回 gcd(a, b) 并通过引用写出 x, y；ceil 为向上取整（处理负数的正确版本）。

```
template <class T> T sign(const T &a) // 符号函数
{
    return a == 0 ? 0 : (a < 0 ? -1 : 1);
}

template <class T> T ceil(const T &a, const T &b) // 向上取整（负数安全）
{
    T A = abs(a), B = abs(b);
    assert(b != 0);
    return sign(a) * sign(b) > 0 ? (A + B - 1) / B : -A / B;
}

int exgcd(int a, int b, int &x, int &y) // 求 gcd(a,b) 与一组解 x, y
{
    if (!b)
    {
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x); // 递归时交换 x/y
    y -= a / b * x;
    return d;
}

auto clac = [&](int a, int b, int c) -> void { // 解 a*x + b*y = c
    int x, y, d = exgcd(a, b, x, y);
    if (c % d != 0)
    {
        cout << -1 << endl; // 无整数解
        return;
    }
    x *= c / d, y *= c / d; // 缩放特解
    int p = b / d, q = a / d, k; // 通解：x + kp, y - kq
    if (x < 0)
    {
        k = ceil(1 - x, p);
        x += p * k;
        y -= q * k;
    }
    else if (x >= 0)
    { // 将x提高到最小正整数
        k = (x - 1) / p;
        x -= p * k; // 将x降低到最小正整数
        y += q * k;
    }
    if (y > 0)
    {                                       // 有正整数解
        cout << (y - 1) / q + 1 << endl;    // 将y减到1的方案数即为解的个数
        cout << x << endl;                  // 当前的x即为最小正整数x
        cout << (y - 1) % q + 1 << endl;    // 将y取到最小正整数
        cout << x + (y - 1) / q * p << endl; // 将x提升到最大
        cout << y << endl;                  // 特解即为y最大值
    }
    else
    {                                       // 无整数解
```

```
        cout << x << endl;                        // 当前的x即为最小的正整数x
        cout << y + q * ceil(1 - y, q) << endl; // 将y提高到正整数
    }
};
```

Miller-Rabin 素性测试 MillerRabin.hpp

- **用法**：is_prime(n)，n 为 long long。
- **复杂度**：单次 O(log^2 n)。
- **注意**：基准 {2,3,5,7,11,13,17,19,23} 对 n < 3.3×10^24 判定正确（覆盖全部 ll）；与 PollarRho.hpp **符号冲突**，同文件二选一。

```
vector<int> Bases{2, 3, 5, 7, 11, 13, 17, 19, 23};

ll mul(ll a, ll b, ll m) // 模乘（__int128 防溢出）
{
    return lll(a) * b % m;
}

ll qmi(ll a, ll b, ll mod) // 快速幂取模
{
    ll res = 1;
    for (; b; b >>= 1, a = mul(a, a, mod))
        if (b & 1)
            res = mul(res, a, mod);
    return res;
}

bool is_prime(ll n)
{
    if (n < 2)
        return false;
    int s = __builtin_ctzll(n - 1); // n-1 = d * 2^s
    ll d = (n - 1) >> s;
    for (auto &p : Bases)
    {
        if (p == n)
            return true;
        ll x = qmi(p, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for (int i = 0; i < s - 1; ++i) // 二次探测链
        {
            x = mul(x, x, n);
            if (x == n - 1)
            {
                ok = true;
                break;
            }
        }
        if (!ok)
            return false; // 该基准判定为合数
    }
    return true;
}
```

Pollard-Rho 质因数分解（含 Miller-Rabin） PollarRho.hpp

- **用法**：factorize(n) 返回升序**带重复**的质因子列表（如 12 → {2, 2, 3}）；is_prime(n) 可单独使用。
- **复杂度**：期望 O(n^{1/4})（每次找一个因子）。
- **注意**：是 MillerRabin.hpp 的超集，符号冲突二选一；n <= 10000 时直接试除；Floyd 判环 + 乘积批量取 gcd 优化。

```
vector<int> Bases{2, 3, 5, 7, 11, 13, 17, 19, 23};

ll mul(ll a, ll b, ll m) // 模乘（__int128 防溢出）
{
    return lll(a) * b % m;
}

ll qmi(ll a, ll b, ll mod) // 快速幂取模
{
    ll res = 1;
    for (; b; b >>= 1, a = mul(a, a, mod))
        if (b & 1)
            res = mul(res, a, mod);
    return res;
}

bool is_prime(ll n) // Miller-Rabin 素性测试
{
    if (n < 2)
        return false;
    int s = __builtin_ctzll(n - 1); // n-1 = d * 2^s
    ll d = (n - 1) >> s;
    for (auto &p : Bases)
    {
        if (p == n)
            return true;
        ll x = qmi(p, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for (int i = 0; i < s - 1; ++i)
        {
            x = mul(x, x, n);
            if (x == n - 1)
            {
                ok = true;
                break;
            }
        }
        if (!ok)
            return false;
    }
    return true;
}

vector<ll> factorize(ll n) // 返回升序带重复质因子
{
    vector<ll> factors;
    auto dfs = [&](auto &&self, ll n) {
        if (n <= 10000) // 小数直接试除
        {
            for (int i = 2; i * i <= n; ++i)
                for (; n % i == 0; n /= i)
                    factors.push_back(i);
            if (n > 1)
                factors.push_back(n);
```

```
            return;
        }
        if (is_prime(n))
        {
            factors.push_back(n);
            return;
        }
        ll c = 1; // 随机迭代函数 (x^2 + c) mod n
        auto rho_func = [&](ll x) { return (mul(x, x, n) + c) % n; };
        ll x0 = 2;
        while (true)
        {
            ll x = x0;
            ll y = x0;
            ll d = 1;
            ll power = 1, lam = 0; // Brent 优化：2^k 步一检查
            ll v = 1;              // 途中乘积
            while (d == 1)
            {
                y = rho_func(y);
                ++lam;
                v = mul(v, abs(x - y), n); // 批量累积差值
                if (lam % 127 == 0)        // 每 127 步取一次 gcd
                {
                    d = gcd(v, n);
                    v = 1;
                }
                if (power == lam) // Floyd 倍增检查点
                {
                    x = y;
                    power *= 2;
                    lam = 0;
                    d = gcd(v, n);
                    v = 1;
                }
            }
            if (d != n) // 找到真因子，分治
            {
                self(self, d);
                self(self, n / d);
                return;
            }
            ++x0; // 该 (x0, c) 失败，换参数重试
            ++c;
        }
    };
    dfs(dfs, n);
    sort(factors.begin(), factors.end());
    return factors;
}
```

线性基 LinearBasis.hpp

- **用法**：LinearBasis<unsigned long long> lb;（建议**无符号整型**）；insert(x) 插入（返回是否新增基）；find(x) 判断 x 能否被表出；max() / min() 最大 / 最低位基；kth(k) 第 k 小可表出值；rebuild() 化为最简标准形。
- **复杂度**：单次 O(BITS)。
- **注意**：kth 从 1 开始计数；插过可被表出的元素（insert 返回 false）后 zero = true（空集 0 可表出），kth 会自动跳过空集；k 超出范围返回 T(-1)。

```
template <typename T> struct LinearBasis
{
    const int BITS = sizeof(T) * 8; // 位宽（如 ull 为 64）
    int sz;       // 基的个数
    bool zero;    // 0 是否可被表出（存在冗余插入）
    vector<T> basis;    // basis[i] 最高位为 i
    vector<T> standard; // 化简后的标准形（升序）

    LinearBasis() : sz(0), zero(false), basis(BITS, T(0))
    {
    }

    // 高斯消元化标准形：basis[i] 只含自己特有的低位
    void rebuild()
    {
        for (int i = BITS - 1; i >= 0; i--)
            for (int j = i - 1; j >= 0; j--)
                if (basis[i] >> j & 1)
                    basis[i] ^= basis[j];

        standard.clear();
        for (int i = 0; i < BITS; i++)
            if (basis[i])
                standard.push_back(basis[i]);
    }

    void clear() // 清空
    {
        sz = 0;
        zero = false;
        basis.assign(BITS, T(0));
        standard.clear();
    }

    bool insert(T x) // 插入 x；返回 false 表示 x 可被现有基表出
    {
        for (int i = BITS - 1; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!basis[i])
                {
                    basis[i] = x, ++sz; // 该位空缺，直接成为基
                    return true;
                }
                x ^= basis[i]; // 消去最高位继续
            }
        zero = true; // x 被完全消成 0
        return false;
    }

    bool find(T x) // x 能否被表出
    {
        for (int i = BITS - 1; i >= 0; i--)
            if (x >> i & 1)
```

```
            {
                if (!basis[i])
                    return false;
                x ^= basis[i];
            }
        return true;
    }

    T max() // 最大可表出值: 贪心
    {
        T res = 0;
        for (int i = BITS - 1; i >= 0; --i)
            res = std::max(res, res ^ basis[i]);
        return res;
    }

    T min() // 最低位的非零基
    {
        for (int i = 0; i < BITS; ++i)
            if (basis[i])
                return basis[i];
        return 0;
    }

    int size() // 基的个数
    {
        return sz;
    }

    T kth(T k) // 第 k 小可表出值 (1-based) ; 越界返回 T(-1)
    {
        if (standard.empty())
            rebuild();
        if (zero)
            --k;      // 跳过空集
        T res = T(0);
        for (auto &x : standard)
        {
            if (k & 1) // k 的第 i 位决定是否取第 i 个基
                res ^= x;
            k >>= 1;
        }
        return k ? T(-1) : res; // k 未用尽: 越界
    }
};
```

计算几何（点 / 线 / 多边形 / 半平面交） Point.hpp

- **用法**：Point<T> 为模板，需自行指定坐标类型并起别名（如 using P = Point<long double>;）后使用；整型坐标（ll）可精确运算（叉积判向），浮点注意精度——本文件不设 eps，所有比较均为精确 ==，故一般优先用整型。
- **内容**：Point（向量运算、流 IO）、Line（两点式）、dot 点积、cross 叉积、length 模长、normalize 单位化、parallel 平行、distancePL 点到直线、distancePS 点到线段、rotate 逆时针旋转 90°、sgn 极角半区、pointOnLineLeft 点在直线左侧、lineIntersection 直线交点、pointOnSegment 点在线段上、pointInPolygon 点在多边形内（含边界）、segmentIntersection 线段相交（返回 {类型, p1, p2}：0 不相交 / 1 严格相交 / 2 重叠 / 3 端点相交）、distanceSS 线段距离、segmentInPolygon 线段在多边形内、hp 半平面交（返回交点列表，逆时针？按排序方向给出多边形顶点）。

```
template <class T>
struct Point { // 二维点 / 向量
    T x;
    T y;
    Point(const T &x_ = 0, const T &y_ = 0) : x(x_), y(y_) {}

    template <class U>
    operator Point<U>() { // 类型转换
        return Point<U>(U(x), U(y));
    }
    // ---- 复合赋值（& 限定：只作用于具名对象，不作用于临时量）----
    Point &operator+=(const Point &p) & { // 向量加
        x += p.x;
        y += p.y;
        return *this;
    }
    Point &operator-=(const Point &p) & { // 向量减
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point &operator*=(const T &v) & { // 数乘
        x *= v;
        y *= v;
        return *this;
    }
    Point &operator/=(const T &v) & { // 数除
        x /= v;
        y /= v;
        return *this;
    }
    // ---- 二元运算 / 取负（值语义，返回新对象）----
    Point operator-() const { // 取负向量
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point &b) { // a + b
        return a += b;
    }
    friend Point operator-(Point a, const Point &b) { // a - b
        return a -= b;
    }
    friend Point operator*(Point a, const T &b) { // 点 * 数
        return a *= b;
    }
    friend Point operator/(Point a, const T &b) { // 点 / 数
        return a /= b;
    }
    friend Point operator*(const T &a, Point b) { // 数 * 点
        return b *= a;
    }
    // ---- 比较与流 IO ----
```

```
    friend bool operator==(const Point &a, const Point &b) { // 逐分量相等
        return a.x == b.x && a.y == b.y;
    }
    friend std::istream &operator>>(std::istream &is, Point &p) { // 读入
        return is >> p.x >> p.y;
    }
    friend std::ostream &operator<<(std::ostream &os, const Point &p) { // 输出 (x, y)
        return os << "(" << p.x << ", " << p.y << ")";
    }
};

template <class T>
struct Line { // 两点式直线 / 线段
    Point<T> a;
    Point<T> b;
    Line(const Point<T> &a_ = Point<T>(), const Point<T> &b_ = Point<T>()) : a(a_), b(b_) {}
};

template <class T>
T dot(const Point<T> &a, const Point<T> &b) { // 点积
    return a.x * b.x + a.y * b.y;
}

template <class T>
T cross(const Point<T> &a, const Point<T> &b) { // 叉积
    return a.x * b.y - a.y * b.x;
}

template <class T>
T square(const Point<T> &p) { // 模长平方
    return dot(p, p);
}

template <class T>
double length(const Point<T> &p) { // 模长
    return std::sqrt(square(p));
}

template <class T>
double length(const Line<T> &l) { // 线段长
    return length(l.a - l.b);
}

template <class T>
Point<T> normalize(const Point<T> &p) { // 单位向量
    return p / length(p);
}

template <class T>
bool parallel(const Line<T> &l1, const Line<T> &l2) { // 平行（含共线）
    return cross(l1.b - l1.a, l2.b - l2.a) == 0;
}

template <class T>
double distance(const Point<T> &a, const Point<T> &b) { // 两点距离
    return length(a - b);
}

template <class T>
double distancePL(const Point<T> &p, const Line<T> &l) { // 点到直线距离
    return std::abs(cross(l.a - l.b, l.a - p)) / length(l);
}

template <class T>
```

```
double distancePS(const Point<T> &p, const Line<T> &l) { // 点到线段距离
    if (dot(p - l.a, l.b - l.a) < 0) { // 在 a 外侧投影
        return distance(p, l.a);
    }
    if (dot(p - l.b, l.a - l.b) < 0) { // 在 b 外侧投影
        return distance(p, l.b);
    }
    return distancePL(p, l); // 垂足落在线段上
}

template <class T>
Point<T> rotate(const Point<T> &a) { // 逆时针旋转 90°
    return Point(-a.y, a.x);
}

template <class T>
int sgn(const Point<T> &a) { // 极角所在半区（上半 / x 正半轴为 1）
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}

template <class T>
bool pointOnLineLeft(const Point<T> &p, const Line<T> &l) { // p 在直线 l 左侧
    return cross(l.b - l.a, p - l.a) > 0;
}

template <class T>
Point<T> lineIntersection(const Line<T> &l1, const Line<T> &l2) { // 直线交点
    return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b));
}

template <class T>
bool pointOnSegment(const Point<T> &p, const Line<T> &l) { // p 在线段 l 上
    return cross(p - l.a, l.b - l.a) == 0 && std::min(l.a.x, l.b.x) <= p.x && p.x <= std::max(l.a.x, l.
b.x)
        && std::min(l.a.y, l.b.y) <= p.y && p.y <= std::max(l.a.y, l.b.y);
}

template <class T>
bool pointInPolygon(const Point<T> &a, const std::vector<Point<T>> &p) { // 点在多边形内（含边界）
    int n = p.size();
    for (int i = 0; i < n; i++) { // 在边上直接算在内
        if (pointOnSegment(a, Line(p[i], p[(i + 1) % n]))) {
            return true;
        }
    }

    int t = 0;                  // 射线（水平向右）穿越计数
    for (int i = 0; i < n; i++) {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line(v, u))) {
            t ^= 1;
        }
        if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line(u, v))) {
            t ^= 1;
        }
    }

    return t == 1; // 奇数次穿越：内部
}

// 线段相交判定，返回 {类型, p1, p2}:
// 0 : 不相交
// 1 : 严格相交（交于内部一点）
```

```
// 2 : 重叠 (p1, p2 为重叠段端点)
// 3 : 相交于端点 (或交点为端点)
template <class T>
std::tuple<int, Point<T>, Point<T>> segmentIntersection(const Line<T> &l1, const Line<T> &l2) {
    // 快速排斥: 包围盒不相交
    if (std::max(l1.a.x, l1.b.x) < std::min(l2.a.x, l2.b.x)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (std::min(l1.a.x, l1.b.x) > std::max(l2.a.x, l2.b.x)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (std::max(l1.a.y, l1.b.y) < std::min(l2.a.y, l2.b.y)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (std::min(l1.a.y, l1.b.y) > std::max(l2.a.y, l2.b.y)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (cross(l1.b - l1.a, l2.b - l2.a) == 0) { // 平行
        if (cross(l1.b - l1.a, l2.a - l1.a) != 0) { // 不共线
            return {0, Point<T>(), Point<T>()};
        } else { // 共线: 求重叠区间
            auto maxx1 = std::max(l1.a.x, l1.b.x);
            auto minx1 = std::min(l1.a.x, l1.b.x);
            auto maxy1 = std::max(l1.a.y, l1.b.y);
            auto miny1 = std::min(l1.a.y, l1.b.y);
            auto maxx2 = std::max(l2.a.x, l2.b.x);
            auto minx2 = std::min(l2.a.x, l2.b.x);
            auto maxy2 = std::max(l2.a.y, l2.b.y);
            auto miny2 = std::min(l2.a.y, l2.b.y);
            Point<T> p1(std::max(minx1, minx2), std::max(miny1, miny2));
            Point<T> p2(std::min(maxx1, maxx2), std::min(maxy1, maxy2));
            if (!pointOnSegment(p1, l1)) {
                std::swap(p1.y, p2.y);
            }
            if (p1 == p2) {
                return {3, p1, p2}; // 交于一点
            } else {
                return {2, p1, p2}; // 重叠
            }
        }
    }
    // 跨立实验: 互相在两侧才相交
    auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
    auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
    auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
    auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);

    if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) || (cp3 < 0 && cp4 < 0)) {
        return {0, Point<T>(), Point<T>()};
    }

    Point p = lineIntersection(l1, l2);
    if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) {
        return {1, p, p}; // 严格相交
    } else {
        return {3, p, p}; // 交在端点
    }
}

template <class T>
double distanceSS(const Line<T> &l1, const Line<T> &l2) { // 线段最近距离
    if (std::get<0>(segmentIntersection(l1, l2)) != 0) {
        return 0.0; // 相交则为 0
    }
```

```
    // 不相交：四个端点到对方线段距离的最小值
    return std::min({distancePS(l1.a, l2), distancePS(l1.b, l2), distancePS(l2.a, l1), distancePS(l2.b,
l1)});
}

template <class T>
bool segmentInPolygon(const Line<T> &l, const std::vector<Point<T>> &p) { // 线段是否完全在多边形内
    int n = p.size();
    if (!pointInPolygon(l.a, p)) {
        return false;
    }
    if (!pointInPolygon(l.b, p)) {
        return false;
    }
    for (int i = 0; i < n; i++) { // 逐边检查与线段的关系
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto w = p[(i + 2) % n];
        auto [t, p1, p2] = segmentIntersection(l, Line(u, v));

        if (t == 1) { // 从内部穿出：不行
            return false;
        }
        if (t == 0) {
            continue;
        }
        if (t == 2) { // 与边重叠：沿多边形方向拐弯则不行
            if (pointOnSegment(v, l) && v != l.a && v != l.b) {
                if (cross(v - u, w - v) > 0) {
                    return false;
                }
            }
        } else { // 交于点：分情况检查是否贴边
            if (p1 != u && p1 != v) { // 交点在边内部：穿出
                if (pointOnLineLeft(l.a, Line(v, u))
                    || pointOnLineLeft(l.b, Line(v, u))) {
                    return false;
                }
            } else if (p1 == v) { // 交点是边的右端点
                if (l.a == v) {
                    if (pointOnLineLeft(u, l)) {
                        if (pointOnLineLeft(w, l)
                            && pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    } else {
                        if (pointOnLineLeft(w, l)
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    }
                } else if (l.b == v) {
                    if (pointOnLineLeft(u, Line(l.b, l.a))) {
                        if (pointOnLineLeft(w, Line(l.b, l.a))
                            && pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    } else {
                        if (pointOnLineLeft(w, Line(l.b, l.a))
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    }
                } else {
```

```
                    if (pointOnLineLeft(u, l)) {
                        if (pointOnLineLeft(w, Line(l.b, l.a))
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    } else {
                        if (pointOnLineLeft(w, l)
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    }
                }
            }
        }
    }
    return true;
}

template <class T>
std::vector<Point<T>> hp(std::vector<Line<T>> lines) { // 半平面交，返回交多边形顶点
    // 按极角排序（同向按叉积，限制在左侧）
    std::sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
        auto d1 = l1.b - l1.a;
        auto d2 = l2.b - l2.a;

        if (sgn(d1) != sgn(d2)) { // 不同半区：上半区在前
            return sgn(d1) == 1;
        }

        return cross(d1, d2) > 0;
    });

    std::deque<Line<T>> ls; // 当前有效半平面
    std::deque<Point<T>> ps; // 相邻半平面交点
    for (auto l : lines) {
        if (ls.empty()) {
            ls.push_back(l);
            continue;
        }

        // 交点已不在新半平面内：弹出队尾
        while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
            ps.pop_back();
            ls.pop_back();
        }

        // 弹出队首
        while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
            ps.pop_front();
            ls.pop_front();
        }

        if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) { // 与队尾平行
            if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) { // 同向：保留更紧的
                if (!pointOnLineLeft(ls.back().a, l)) {
                    assert(ls.size() == 1);
                    ls[0] = l;
                }
                continue;
            }
            return {}; // 反向平行：无交
        }

        ps.push_back(lineIntersection(ls.back(), l));
```

```
        ls.push_back(l);
    }

    // 收尾：队首半平面检查队尾交点
    while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
        ps.pop_back();
        ls.pop_back();
    }
    if (ls.size() ⩽ 2) { // 不足三条：无有效交
        return {};
    }
    ps.push_back(lineIntersection(ls[0], ls.back())); // 闭合多边形

    return std::vector(ps.begin(), ps.end());
}
```